

型付き AI エージェント言語 AIPL の 設計と実現とその実行環境について

児玉靖司 (Yasushi Kodama)

法政大学 yass@hosei.ac.jp

2026-06-12

Abstract

大規模言語モデル (LLM) を中核に据えた AI システムは、プランナ・ソルバ・レビューといった複数の自律エージェントが非同期に協調するアーキテクチャへ急速に収束しつつある。本稿は、この構造を言語仕様として第一級に表現する **型付き AI エージェント言語 AIPL** (Actor-based Intelligent Parallel Language) の設計・実現・実行環境を一貫して記述する。AIPL は 1973 年 Hewitt のアクターモデルと 1986 年の ABCL/1 の系譜を継ぎ、そこへ (i) 注釈を一切要求しない **Hindley–Milner (HM) 型推論**, (ii) capability 効果・線形・セッションといった **段階的型機能**, (iii) LLM 呼び出しを言語プリミティブ `ai_call` として埋め込む **第一級 AI 連携** の三層を重ねたものである。実現面では、同一の `.abcl` ソースに対して **7 種のランタイム** (注釈版/推論版 Python, OCaml 正典, ブラウザ/Node の JavaScript, C) と **9 種のコード生成バックエンド** (C+pthread, SDL2, Xinu, Python, Pony, Erlang, Go, Prolog, LLVM/OpenMP) を実装し、そのうち HM 推論を備える 3 実装 (OCaml, 推論版 Python, C コード生成) が **同一ソースに対し完全一致する推論型** を産出することを経験的に検証した。実行環境としては、1:1 OS スレッド・M:N 軽量スレッド・協調イベントループの 3 並行モデルを同一ソースから選択可能とし、さらにトークン予算管理・チェックポイント・隔離・CRDT 状態・saga 補償からなる分散ランタイム層 (DR-1..13) と Mac と Raspberry Pi 上のベアメタル OS Xinu を跨ぐ実機分散実行を実現した。型推論の cross-validation, 型健全性の局所性解析, そして 213 超のスモークテスト/デモが全て green である点を評価として報告する。

Contents

1	はじめに	2
1.1	背景と動機	2
1.2	本稿の貢献	3
1.3	構成	3
2	背景と関連研究	3
3	AIPL の言語設計	4
3.1	計算モデル	4
3.2	三種の送信	4
3.3	become と select	4
3.4	チャンネルと構造化並行	4
3.5	LLM 連携プリミティブ	5
4	型システムと型推論	5
4.1	型の世界	5
4.2	Hindley–Milner 型推論	5
4.2.1	2 パス推論アルゴリズム	6
4.2.2	単一化の主要規則	6
4.3	段階的型付けの第二軸	6
4.4	型健全性	6

5	実現:7 ランタイムと 9 バックエンド	7
5.1	多実装アーキテクチャ	7
5.2	パース機構	7
5.3	HM 駆動のコード特殊化	7
5.4	9 種のコード生成バックエンド	7
6	実行環境 (1): 並行モデルと LLM 連携	7
6.1	3 並行モデルの統一	7
6.2	LLM ガバナンス	8
6.3	WebSocket とダッシュボード	8
7	実行環境 (2): 分散ランタイムと実機分散	8
7.1	分散ランタイム DR-1..13	8
7.2	HTTP/JSON 分散とリモート送信	8
7.3	Mac × Raspberry Pi (Xinu) 実機分散	9
7.4	進化的言語設計パイプライン	9
8	ケーススタディ	9
8.1	LLM マルチエージェント・パイプライン	10
8.2	Mac × Xinu 実機分散:5 人の哲学者	10
9	評価	10
9.1	推論の cross-validation	10
9.2	機能パリティとテスト	10
9.3	セルフホスト	11
9.4	実装規模	11
10	議論と限界	11
11	結論	11

1 はじめに

1.1 背景と動機

LLM の能力向上にともない、実務的な AI システムは単一のモデル呼び出しではなく、役割の異なる複数エージェント (プランナ、ソルバ、レビューア、ツール実行器など) がメッセージを介して非同期に協調する構成に収束しつつある。これは奇しくも、1986 年に米澤明憲らが提唱したオブジェクト指向並行計算モデル **ABCL/1** [2] が描いた未来像——独立した状態を持つオブジェクトが過去型・現在型・未来型の三種のメッセージ送信で協調する——と本質的に同型である。しかし当時のアクター言語には、現代の AI エージェントが必要とする トークン予算管理・タイムアウト・プロバイダ・フォールバック・クロスマシン協調を言語仕様で吸収する仕組みが存在しなかった。

本研究は、この欠落を埋める言語として **AIPL** (Actor-based Intelligent Parallel Language) を設計・実現した。AIPL は ABCL の直系再実装を基盤としつつ、

1. 型注釈フリーなアクター言語を Hindley–Milner 型推論で実現し、
2. LLM 呼び出しを `ai_call(provider, prompt)` という言語プリミティブとして第一級化し、
3. 同一ソースから OS スレッド・軽量スレッド・イベントループの 3 並行モデル、そして分散・実機実行までを統一的に駆動する、

ことを目指す。AIPL は単なる処理系ではなく、“AI-OS のスクリプト層”——すなわち多数の AI エージェントの並行性・永続性・予算・クロスマシン協調をランタイムが統治するための言語層——として位置づけられる。

1.2 本稿の貢献

本稿の貢献は次の四点である.

(C1) 型注釈フリーなアクター言語の実現

表面構文に型注釈を一切要求せず, フィールド型・メソッドシグネチャを呼び出しサイトと初期化式から自動推論するアクター言語を設計し, HM を備えた 3 つの独立実装が同一ソースに対し 同一の推論結果を産出することを経験的に検証した (§9).

(C2) HM 駆動のコード特殊化

推論結果を活用し, アクター間メッセージ境界以外の領域を `value_t` タグ付き共用体からネイティブ C 型へ 特殊化するコード生成手法を確立した (§5.3).

(C3) AI 連携の第一級化

ABCL/1 の三種送信のいずれからとも LLM を駆動でき, トークン予算・並行度・フォールバック・利用量記録を環境変数とランタイムで統治する枠組みを構築した (§6.2).

(C4) 単一ソース・多並行モデル・分散実行

同一の `.abcl` ソースを 3 並行モデルへ無改変配置し, さらに HTTP/JSON および UART/TCP 経由で Mac と Raspberry Pi 上のベアメタル OS Xinu を跨ぐ実機分散実行を実証した (§6,§7).

1.3 構成

§2 で背景と関連研究を整理し, §3 で言語設計を, §4 で型システムと型推論を述べる. §5 で 7 ランタイムと 9 バックエンドの実現を, §6–§7 で実行環境 (並行モデル・LLM 連携・分散ランタイム・進化的パイプライン) を述べ, §9 で評価, §10 で議論と限界, §11 で結論を述べる.

2 背景と関連研究

アクターモデルと ABCL. アクターモデルは Hewitt [1] に始まり, 各アクターが (i) メッセージ受信, (ii) 内部状態の更新, (iii) 新たなアクター生成・送信を行う計算単位とする. 米澤らの ABCL/1 [2] は, これに 過去型 (*past*, *fire-and-forget*), 現在型 (*now*, 同期), 未来型 (*future*, 非同期ハンドル) の三種送信を導入し, オブジェクト指向並行計算の古典となった. AIPL は ABCL/1 の三種送信をそのまま継承する (§3).

Hindley–Milner 型推論. Hindley [3] と Milner [4] に始まる HM 型システムは, Damas–Milner [5] の Algorithm W により, 注釈なしに主要型 (principal type) を推論できることが保証される. AIPL の推論器は `unify / generalize / instantiate` と `let` 多相からなる教科書的 HM を採用する (§4.2).

段階的型付け・効果・線形・セッション. Siek–Taha の gradual typing [6] は静的型と動的型の連続的な共存を可能にする. AIPL は推論版を主軸としつつ, 注釈版で capability 効果 [7], 線形型 [8], セッション型 [9] を参照実装として保持し, 両者を一言語の二軸 (推論的 HM 軸と注釈的段階的軸) として統合する (§4.3).

構造化並行と分散. `scope { ... }` による構造化並行は Java Project Loom の StructuredTaskScope [10] と意味論を同じくする. 分散層では CRDT [11] によるアクター状態複製, saga パターンによる補償トランザクションを取り込む.

進化的言語設計. AIPL の次世代機能群は, MAP-Elites [12] に基づく進化計算で言語設計の軸を経験導出し, その勝者を機能として実装する, という独自のループ (§7.4) を経て決定された.

既存のアクター/エージェント基盤との比較. Erlang/OTP,Akka,Pony,Ray などは成熟したアクター/分散基盤だが, LLM 連携・型推論・コード生成多様性の観点では設計目標が異なる(表 1).AIPL の独自性は,(i) 注釈フリー HM 推論をアクター言語に適用した点,(ii) 単一フロントエンドから 9 バックエンドへ射影する点,(iii) LLM 呼び出しとトークン予算を言語/ランタイムで第一級に統治する点, の三つの同時実現にある.

Table 1: 既存基盤との設計目標比較

	三種送信	型推論	多バックエンド	LLM 第一級	主たる並行モデル
Erlang/OTP	△	×	×	×	M:N (BEAM)
Akka (Scala)	△	△	×	×	M:N (JVM)
Pony	×	△	×	×	M:N (参照能力)
Ray (Python)	△	×	×	△	1:1 + クラスタ
AIPL	✓	✓	✓	✓	1:1 / M:N / event-loop

3 AIPL の言語設計

3.1 計算モデル

AIPL のプログラムは `class` 宣言の集合である. 各 `class` は一つのアクター種を定義し,`new C(args)` でインスタンス (アクター) を生成する. 各アクターは独立した **mailbox** を持ち, フィールド (状態) はアクター内に閉じて外部から直接 `read/write` できない. アクター間の唯一の相互作用はメッセージ送信である.

3.2 三種の送信

ABCL/1 を継承し,AIPL は次の三種の送信を持つ (リスト 1).

Listing 1: 三種送信と reply

```
class Counter {
  var count = 0;
  method tick() { count = count + 1; print("count = " + count); }
  method get() { reply(count); } // 同期送信への応答
}
var c = new Counter();
send c.tick(); // 過去型 (past) : fire-and-forget
var v = now c.get(); // 現在型 (now) : reply 到着までブロック
var f = future c.get(); // 未来型 (future) : 非同期ハンドル
var w = await(f); // ハンドルから値を取得
```

`reply(value)` は `now/future` 送信の `reply` スロットを充足する. `future_done(f)` で非ブロッキングにポーリングでき, `await(f)` でブロックして値を取得する. メソッド本体内では特殊変数 `self` (自アクター) と `sender` (送信元) が利用可能である.

3.3 become と select

`become C(args)` はアクターのクラスとフィールドを実行時に差し替える (状態遷移の表現). `select { case ... }` は複数チャネル/メッセージからの選択受信を行い,Erlang の `receive` に相当する.

3.4 チャンネルと構造化並行

`channel[T] / channel[T, cap=N]` は CSP 風の型付きチャンネルを提供する (`channel_send / channel_recv / select_recv`). `scope { ... }` ブロックは, その内部で生成された未来 (`future`) をブロック脱出時に

自動 join する構造化並行構文であり, 親スコープが子タスクのライフタイムを所有することを保証する.

3.5 LLM 連携プリミティブ

LLM 呼び出しは `ai_call([provider,] prompt)` として言語に組み込まれている. プロバイダ ID は 1=Gemini (既定) / 2=Anthropic (Claude) / 3=OpenAI / 0=auto であり, 環境変数 `AIPL_AI_PROVIDER=mock` で全呼び出しをオフライン化できる. 三種送信のいずれからも LLM を駆動できる (リスト 2).

Listing 2: LLM × 未来型による並列問い合わせ

```
class Asker { method ask(q) { var r = ai_call(2, q); reply(r); } }
var a = new Asker();
var f1 = future a.ask("Translate hello to French.");
var f2 = future a.ask("Translate hello to German.");
print(await(f1)); // 2 つの LLM 呼び出しが並列に走る
print(await(f2));
```

4 型システムと型推論

4.1 型の世界

AIPL の型は次のとおりである. 基本型 `int` (64-bit), `float` (倍精度; 100. のように末尾ドット可), `string` (UTF-8), `bool`, `unit`; 構造型 `actor(C)` (クラス `C` のインスタンス), `array`, `tuple(...)` (位置・不変・固定アリティ; 長さ各スロット型が型の一部), `record{a:int, b:string}` (構造的・名前付きフィールド), `channel[T]`, 関数型, `future`; そして単一文字の型変数 (`T U V ...`) による暗黙多相である. ジェネリック関数は `function id(x:T)->T` のように書ける.

4.2 Hindley–Milner 型推論

AIPL の正典実装 (OCaml) の推論器 `infer.ml` は教科書的 HM を実装する: mutable union-find による型変数, `unify / generalize / instantiate`, `let` 多相からなる. クラスのメソッドは仮の型変数で事前登録され, 本体検査でボトムアップに具体化される **2 パス方式**をとる. これにより, 呼び出しサイトからの情報がメソッドシグネチャを自動的にモノモルフィズする.

Listing 3: 注釈なし = 全推論. 呼び出しサイトが型を確定する

```
class Hello {
  var count = 0.; // 0. リテラルから count:float
  method init(n) { // n は注釈なし
    count = n; // count:float なので n:float に確定
    print("hi " + n);
  }
  method greet() { print("count = " + count); }
}
var h = new Hello(5); // init 引数型を呼び出しサイトから再確定
```

リスト 3 に対し, 推論器は `count : float, init : (int) -> unit, greet : () -> unit` を導く. `Counter.dec(x)` のようなメソッドは宣言時には $\forall \alpha. \alpha \rightarrow unit$ だが, 本体 `count - x (count:float)` によって $\alpha = float$ に固定され, 別途モノモルフィゼーション・パスを要さない (HM の `unify` が自動的に果たす). なお高階多相 (rank-2 以上) は持たず, 関数を引数に取る位置は any 扱いとする.

4.2.1 2 パス推論アルゴリズム

アクター言語に HM を適用する際の中心的課題は、メソッド間の相互参照とフィールドの共有である。クラス c のメソッド m が同じクラスの別メソッド n を `self.n(...)` で呼ぶとき、 n の型が未確定でも m を検査できねばならない。AIPL はこれを 2 パスで解く。

1. **パス 1 (事前登録):** クラス内の各フィールドに新鮮な型変数を、各メソッドに $\alpha \rightarrow \beta$ 形の仮シングネチャ (引数・戻り値とも新鮮な型変数) を割り当て、型環境 Γ に登録する。
2. **パス 2 (本体検査):** 各メソッド本体をボトムアップに検査し、代入・呼び出し・演算が生む等式制約を `unify` で解く。フィールドはメソッドを跨いで共有されるため、あるメソッドでの使用が他メソッドの型を確定させる。

パス 2 の単一化は標準的な occurs-check 付き union-find であり、`instantiate` は `let` 束縛の汎化済みスキームを新鮮化し、`generalize` は環境に自由でない型変数を $\forall$ で閉じる。リスト 3 では、パス 1 で `init` を $(\alpha) \rightarrow \text{unit}$ とし、パス 2 で本体 `count = n` (ただし `count : float`) から $\alpha = \text{float}$ を導く。さらに `new Hello(5)` の呼び出しサイトが整数リテラル 5 を与えるため、最終的に `init` 引数は `int` に再確定される。

4.2.2 単一化の主要規則

単一化 $\text{unify}(\tau_1, \tau_2)$ は次の規則に従う (抜粋)。 σ は型変数代入、 $\doteq$ は被単一化対を表す。

$$\begin{aligned}\alpha \doteq \tau &\Rightarrow [\alpha \mapsto \tau] \quad (\alpha \notin \text{ftv}(\tau) \text{ occurs-check}) \\ (\tau_1 \rightarrow \tau_2) \doteq (\tau'_1 \rightarrow \tau'_2) &\Rightarrow \text{unify}(\tau_1, \tau'_1); \text{unify}(\tau_2, \tau'_2) \\ \text{actor}(C) \doteq \text{actor}(C') &\Rightarrow C = C' \\ \text{record}(R_1) \doteq \text{record}(R_2) &\Rightarrow \text{共通フィールドを単一化 (width subtyping)} \\ \text{any} \doteq \tau &\Rightarrow \text{成功 (gradual 境界, transient cast を挿入)}\end{aligned}$$

最後の `any` 規則が段階的型付けとの接合点であり、§4.4 の健全性境界を生む箇所である。

4.3 段階的型付けの第二軸

AIPL は推論版を主軸としつつ、注釈版を参照実装として保持し、段階的型付けの諸機能を提供する。

- **capability 効果:** `function read_url(u:string)->string !{net,fs}{...}` のように、メソッド型に $\{ai, fs, net, mut\}$ の効果行を付す。43 個の組込関数に既定効果 `BUILTIN_EFFECTS` を割当て、効果を推論する。
- **線形型:** `var fh: linear int = open("x");` のように、リソースの一回使用を静的に保証する。
- **所有:** `pub var holder: string = "";` で公開フィールドを区別する。
- **セッション型:** `session_define("Solve", "solver.solve(string) ! string -> reviewer.review(...) -> ...")` のように、アクター間プロトコルを動的に定義する。
- **refinement:** 整数・実数・有理数に対する述語制約を SMT ソルバ Z3 へ委譲して検査する (宣言時に `vacuously-false` を検出する)。

これらは推論版にはランタイム経由でのみ存在し、注釈版が静的検査の参照実装を担う、という二軸構造をとる。

4.4 型健全性

型健全性は `progress` と `preservation` の二命題で論じられる。

定義 1 (Progress). 型付け済みプログラムの実行は、型不整合 (例: 文字列に対する数値演算) によって *stuck* しない。

定義 2 (Preservation). 評価ステップを進めても、値の型は宣言型と (*any* を含む単純な部分順序の下で) 互換である。

AIPL は段階的型付け言語であるため、典型的な 局所健全 (*local soundness*) だが大域非健全 (*global unsoundness*) の特性を持つ。健全性が崩れうる境界は次の四つである: (1) 未注釈領域から注釈領域への *any* 流入, (2) 動的反射 (*compile / become / add_method*) によるクラス形状の変化, (3) 動的ディスパッチで送信先シグネチャが乖離する場合, (4) *record* 経由でアクターフィールドを書き換える場合。最優先の *unsoundness gap* である *any*→*T* 境界に対しては, *--transient* フラグによる **transient cast** を導入し, 変数宣言・メソッド引数・関数引数の 3 境界で, 静的検査の互換規則と同一規則のランタイムチェックを挿入して塞いだ。

5 実現:7 ランタイムと 9 バックエンド

5.1 多実装アーキテクチャ

AIPL は単一のソース言語に対し **7 種のランタイム実装**を持つ (表 2)。このうち HM 推論を採用するのは OCaml・推論版 Python・C コード生成の 3 実装である。

Table 2: AIPL の 7 ランタイム実装

短縮名	ソース	型システム
Py-A (注釈版)	python-aipl/	段階的型付け (Phase 11–17 参照実装, 最も機能完備)
Py-I (推論版)	python-aipl-inferred/	HM 推論 (interp は Py-A を共有, 899 行)
OCaml (正典)	src/*.ml	HM 推論 (infer.ml 484 行, WebSocket 内蔵)
JS-O (OCaml 経由)	src/app.js + gateway	= OCaml HM (/api/typecheck)
JS-B (ブラウザ)	browser-abcl/	flow-sensitive 軽量推論 (typecheck.js 308 行)
JS-N (Node)	node-aipl-server/	flow-sensitive (継承), HTTP wrap
C (aipl2c)	aipl2c.ml + 各 C runtime	HM + コード特殊化

5.2 パース機構

OCaml 正典実装は *lexer.mll* (*ocamllex*) + *parser.mly* (*Menhir*) で字句・構文解析を行い, *infer.ml* → *eval_thread.ml* へ流す。Python 実装は Lark 文法 (*grammar.lark*) を用い, AST を *aipl_typeck / aipl_inference* へ渡す。ブラウザ/Node 実装は *browser-abcl* の自前パーサを共有する。

5.3 HM 駆動のコード特殊化

C バックエンド (*aipl2c*) の中核技術は, HM 推論結果を用いた**コード特殊化**である。 *l_a*, *l_b* が *int* と推論されれば, *value_t* タグ付き共用体演算 *v_binop*("+", *l_a*, *l_b*) ではなく, ネイティブな *long l_x = (l_a + l_b);* を発行する。 *value_t* はアクター間メッセージ境界 (*value_t args[]*) のみで *uniform* に保ち, アクター内部は特殊化する。これにより *mailbox* の一様性と内部の静的特殊化を両立する。

5.4 9 種のコード生成バックエンド

OCaml 製の *aipl2c* は単一フロントエンドから複数のバックエンドへコードを生成する (表 3)。送信操作は各ターゲットの並行プリミティブへ射影される: C+*pthread* は *enqueue(...)*, Pony は *obj.method(args)*, Erlang は *Pid ! {method, Args}*, Go は *obj.mailbox <- msgT{...}*, Prolog は *thread_send_message(...)* である。Pony/Go/Erlang では構築と *init* 本体を分離する二段階初期化をとる。

6 実行環境 (1): 並行モデルと LLM 連携

6.1 3 並行モデルの統一

AIPL の重要な実証は, 同一の *.abcl* ソースを無改変で 3 つの並行モデルへ配置できる点である。

Table 3: 単一フロントエンドからの 9 バックエンド

フラグ	ターゲット	並行モデル
(既定)	C + pthread	1:1 OS スレッド
-lSDL2	C + SDL2 GUI	1:1 (描画付き)
--xinu	Xinu C	ベアメタル協調
--python	Python	1:1 OS スレッド
--pony	Pony	M:N 軽量
--erlang	Erlang/BEAM	M:N 軽量
--go	Go goroutine	M:N 軽量
--prolog	SWI-Prolog	スレッド
--llvm / --openmp	LLVM / OpenMP	ホット/コールド注釈・並列 for

1. 1:1 OS スレッド (Python/OCaml/C): ~ 100 – 1000 アクター規模.
2. M:N 軽量スレッド (Pony/Erlang/Go codegen): $\sim 10^5$ – 10^7 アクター規模.
3. 協調イベントループ (browser-abcl/node): 単一スレッドの `setTimeout(0)` スケジューリング.

アクター抽象が並行モデルから独立しているため, プロトタイプ (イベントループ) から高スループット (M:N) まで, ソースを変えずに移行できる.

6.2 LLM ガバナンス

ai_call 系プリミティブ (`ai_call_with_system`, `ai_call_retry`, `ai_call_image`, `ai_call_priority`) は, ランタイムが次の環境変数で統治する: `ABCL_AI_TOKEN_BUDGET` (超過で `BudgetExceeded`), `ABCL_AI_MAX_CONCURRENT` (in-flight セマフォ), `ABCL_AI_FALLBACK_MODELS` (フォールバック連鎖), `ABCL_AI_USAGE_FILE` (利用量記録), `AIPL_AI_PROVIDER=mock` (オフライン smoke). バックエンドは Ollama (ローカル), OpenAI, Gemini, Anthropic に対応する.

6.3 WebSocket とダッシュボード

全 7 ランタイムが統一ワイヤプロトコルの WebSocket を備える: OCaml は `web_gateway.ml` に RFC 6455 を自前実装 (40 行), Python は `aipl_websocket.py`, JS-B はブラウザネイティブ, JS-N は `ws`, C は `libwebsockets` である. クライアントは `?sid=<group>` で参加し, `{"kind": "welcome", "sid": "...", "peers": N}` を受信する. `aipl_main.py --dashboard 8800` はライブカウンタ・観測トラフィック表・SSE イベントストリームを提供し, `ABCL_PEER_DASHBOARDS` で複数 worker を TOTAL 行に集約する.

7 実行環境 (2): 分散ランタイムと実機分散

7.1 分散ランタイム DR-1..13

`AIPL_DIST_ENABLE=1` で有効化される分散ランタイム層 (`aipl_dist`) は, AI エージェント運用に必要なガバナンス機能を opt-in で提供する (表 4).

7.2 HTTP/JSON 分散とリモート送信

リモート送信は HTTP/JSON ワイヤプロトコルで実現される: `POST /api/json/send` (fire-and-forget) と `POST /api/json/call` (同期 reply). Python では `remote_call` / `remote_now` / `remote_future`, `web_listen` (port) / `web_expose` / `serve_forever` を提供する. `ABCL_REMOTE_SECRET` を設定すると HMAC-SHA256 署名 (`x-ABCL-Sig`) を必須化し, Python/OCaml/C 間でワイヤ互換である (OCaml は純正 `hmac_sha256.ml`). これにより, プロバイダの異なる 3 ノード (`coordinator` / `solver` / `verifier`) を跨ぐ協調が可能となる.

Table 4: 分散ランタイム機能 DR-1..13

ID	機能	概要
DR-1	env-var ルーティング	AIPL_ROUTE, アクター名 →tag テーブル
DR-2	構造化ログ	ND-JSON 1 行/event
DR-3	トークン予算協調	60 秒スライディング窓 (RPM/TPM)
DR-4	checkpoint/resume	atomic save/restore
DR-5	隔離とスキップ	自動 quarantine (TTL)
DR-6	quorum 複製	並列 multi-provider
DR-7	部分木隔離	OTP 的 blast-radius 限定
DR-8	spawn-tree 追跡	TLS-auto parent
DR-9	ランタイムフック	interp/actor/call_ai (opt-in)
DR-10	CRDT 状態	G-Counter/OR-Set/LWW-Register (15 prims)
DR-11	saga 補償	saga{step{}}compensate{}} (LIFO)
DR-12	多リージョン failover	per-region route + chain walk
DR-13	自動スケール pool	hysteresis 駆動 (pool_create 等)

7.3 Mac × Raspberry Pi (Xinu) 実機分散

さらに AIPL は,Mac 上のホストアクターと Raspberry Pi 上のベアメタル OS **Xinu** 上の AIPL アクターを跨いだ実機分散を実証した. 両者は UART/TCP 上の行指向 RPC (opcode: PING/LIST/SEND/QUERY/LOAD/COMPILE/RUN/SPAWN) で接続され, 推論版 Python は uart1:// スキームでこれを透過的に送信に翻訳する. 代表例として「分散 5 人の哲学者」を Mac 3 ノード + Xinu 2 ノードで構成し,5 本のフォークを Xinu アクター化して境界を跨いだ相互排他同期を実機で達成した.

7.4 進化的言語設計パイプライン

AIPL の次世代機能群 (CE-1..13 の型推論強化,DR-1..13 の分散機能) は, aice-evolution-v2 の **MAP-Elites** 遺伝的アルゴリズムが言語仕様 (.aice) を入力に cross-task 評価と LLM 誘導突然変異を行い,“言語設計の普遍軸”を経験導出した結果として実装された. たとえば構造化並行 (scope{future...}) は, 進化計算が concurrency=structured * ownership=linear * type=dependent を上位軸として pin した予測に基づいて言語機能化された. パイプラインは .aice → .ga.json → .abcl の三段からなる. 表 5 に, 進化計算が導出し実装へ反映した予測軸を示す.

Table 5: 進化計算が導出した言語軸とその実装

予測 (軸)	勝者の値	実装
type_safety	dependent	refinement + Z3 (CE-6/CE-9)
effect	capability	capability 効果行 (CE-10/CE-11)
concurrency	csp_channels	channel[T] (CE 系 / Phase 13)
ownership	linear	線形型 linear
state	symbol_owned	アクター閉包フィールド
concurrency (再)	structured	scope{future...}

勝者個体 (balanced / hang-resilience / Erlang-OTP-like) は, 8 seed × 30 世代 × 2 run (各約 24 分,1400 超の LLM 呼び出し) の MAP-Elites 探索から選抜された. これらが DR-1..13 の実装母体となった.

8 ケーススタディ

本節では,AIPL の設計が実アプリケーションにどう結実するかを, 二つのケーススタディで示す.

8.1 LLM マルチエージェント・パイプライン

プランナ・ソルバ・レビューアの3エージェントが協調して問題を解く構成を、三種送信と `ai_call` で簡潔に記述できる (リスト 4). 未来型送信により `solver` と `reviewer` の LLM 呼び出しを重ね合わせ、`scope` で子タスクのライフタイムを束ねる.

Listing 4: `planner→solver→reviewer` の協調

```
class Solver { method solve(q) { reply(ai_call(2, "Solve: " + q)); } }
class Reviewer { method review(q, a) { reply(ai_call(2, "Grade:" + q + a)); } }
class Planner {
  var s = new Solver();
  var r = new Reviewer();
  method run(q) {
    scope {
      var fa = future s.solve(q); // solver の LLM 呼び出し
      var a = await(fa);
      var fr = future r.review(q, a); // reviewer の LLM 呼び出し
      reply(await(fr));
    } // scope 脱出時に未完了 future を auto-join
  }
}
var p = new Planner();
print(now p.run("integral of x^2"));
```

トークン予算 (`ABCL_AI_TOKEN_BUDGET`), 並行度 (`ABCL_AI_MAX_CONCURRENT`), quorum 複製 (DR-6), フォールバック (`ABCL_AI_FALLBACK_MODELS`) は, このソースを一切変えずに環境変数とランタイムで統治される.

8.2 Mac × Xinu 実機分散:5 人の哲学者

古典的な食事する哲学者問題を, 5 本のフォークを Raspberry Pi 上の Xinu アクターとして配置し, 哲学者を Mac 上のアクターとして配置した混在トポロジで実行した. Mac 側の推論版 Python は, フォークアクターへの送信を `uart1://` スキームの行指向 RPC (QUERY/SEND) へ透過翻訳し, 境界を跨いだ相互排他同期を実機で達成した. 同一の哲学者ロジックを, (a) 単一プロセス 1:1 スレッド, (b) Go goroutine codegen による M:N, (c) Mac+Xinu の実機分散, の三形態へ無改変で配置できる点が, §6 の並行モデル独立性を具体的に裏づける.

9 評価

9.1 推論の cross-validation

AIPL の主要な経験的主張は, HM を備える 3 実装 (OCaml / 推論版 Python / C コード生成) が同一 `.abcl` ソースに対し完全一致する推論型を産出することである. 代表例を以下に示す (3 実装すべてで一致):

```
Hello: count : float; init : (int) -> unit;
       greet : () -> unit; inc : () -> unit
Counter: count : float; inc : () -> unit;
       dec : (float) -> unit <- monomorphized via call site
```

9.2 機能パリティとテスト

次世代機能群 CE-1..13 + DR-1..13 は, 主要ランタイム間で機能同等 (parity) が確認されている. Phase O 完了時点で, 分散 DR-1..9 と型推論 CE-1..9 が Py-A / OCaml / JS-O の各実装で 9/9,

合計 18/18 (実装 ~1305 LOC) であった. JS-B / JS-N も CE-10/CE-12/CE-13/DR-11 を含めて 26/26 で closeout した. 表 6 にスモークテストの集計を示す.

Table 6: スモークテスト集計 (代表値)

項目	結果
OCaml ランタイム	52/52
ブラウザ (構文/パース)	7/7 + 4/4
Python (.abcl / 旧 .aipl)	33/33 + 21/21
分散クロス言語 (3-node mock)	8/8
セルフホスト塔 (10 段階, 41 samples)	47/47
aipl_dist 単体	27/27
分散 MVP デモ (8 feature × 3)	24/24
next-gen / self-host アサーション	103 全 PASS
合計	213+ テスト/デモが green

9.3 セルフホスト

AIPL を AIPL 自身で記述する aipl-self-host/ の塔は, 10 段階 (Level A-C-3 と統合 Level Z) からなり, 47/47 のスモークがすべて green である. Level Z は 18 行の Python bootstrap と CE-11 capability-checked な io_bridge.abcl による統合フルセルフホストであり, 各 Level の checker を自身に適用して phase monotonicity を実証している.

9.4 実装規模

主要コンポーネントの規模は次のとおりである: OCaml HM ≈985 行 (infer.ml 484 + types.ml + typing_env.ml), 推論版 Python 899 行, aipl_inference.py ~1255 行, aipl_dist.py 591 行, typecheck.js 308 行, WebSocket 40 行, C+SDL2 ランタイム 1178 行, セルフホスト 9 モジュール ~1500 行.

10 議論と限界

健全性の限界. §4.4 で述べたとおり, AIPL は段階的型付け言語であり, 注釈領域については局所健全だが, 未注釈領域・動的反射との境界では大域健全性が成り立たない. transient cast は最優先 gap を塞ぐが, ランタイム型キャストは限定的で, 完全な transient soundness には届かない弱い保証である. 動的反射 (become / compile) を含むプログラムの健全性保証は今後の課題である.

推論の表現力. 現状の HM は高階多相 (rank-2 以上) を持たず, 関数を引数に取る位置は any に退避する. また narrowing は単純条件に限られ, && 連鎖やネストした条件の絞り込みは未対応である.

版差. 本稿の数値は時期の異なる複数のレポートから統合しており, ランタイム数 (7 vs 6, Py-A の数え方), セルフホスト段階数 (9/37 vs 10/47) などに版差がある. 本稿では最新の機能マトリクス (10 段階・47/47・7 ランタイム) を採用した.

今後の課題. (i) 動的反射を含む健全性保証の強化, (ii) 高階多相と本格的な flow-sensitive narrowing, (iii) M:N バックエンドでの大規模 (>10⁶ アクター) 実機評価, (iv) 進化的パイプラインによる言語機能予測の定量検証, が挙げられる.

11 結論

本稿は, LLM 駆動 AI システムの自然なアーキテクチャである “協調する複数エージェント” を言語仕様に第一級表現する型付き AI エージェント言語 AIPL の, 設計・実現・実行環境を一貫

して述べた. AIPL は ABCL/1 の三種送信に HM 型推論と段階的型機能, そして `ai_call` による第一級 AI 連携を重ね, 7 ランタイム・9 バックエンド・3 並行モデルを単一ソースから駆動し, HTTP/JSON および UART/TCP を介して Mac と Raspberry Pi (Xinu) を跨ぐ実機分散までを実現した. HM を備える 3 実装が同一ソースに対し完全一致する推論型を産出すること, そして 213 超のテスト/デモが green であることを経験的に示した. AIPL は, アクター言語の古典的理想と現代の AI エージェント運用を橋渡しする “AI-OS のスクリプト層” としての一つの到達点を提供する.

References

- [1] C. Hewitt, P. Bishop, R. Steiger. A Universal Modular ACTOR Formalism for Artificial Intelligence. *IJCAI*, 1973.
- [2] A. Yonezawa, J.-P. Briot, E. Shibayama. Object-Oriented Concurrent Programming in ABCL/1. *OOPSLA*, 1986.
- [3] J. R. Hindley. The Principal Type-Scheme of an Object in Combinatory Logic. *Trans. AMS*, 1969.
- [4] R. Milner. A Theory of Type Polymorphism in Programming. *J. Computer and System Sciences*, 1978.
- [5] L. Damas, R. Milner. Principal Type-Schemes for Functional Programs. *POPL*, 1982.
- [6] J. G. Siek, W. Taha. Gradual Typing for Functional Languages. *Scheme Workshop*, 2006.
- [7] A. Mettler, D. Wagner, T. Close. Joe-E: A Security-Oriented Subset of Java. *NDSS*, 2010.
- [8] P. Wadler. Linear Types Can Change the World! *Programming Concepts and Methods*, 1990.
- [9] K. Honda, V. T. Vasconcelos, M. Kubo. Language Primitives and Type Discipline for Structured Communication-Based Programming. *ESOP*, 1998.
- [10] R. Pressler et al. JEP 453: Structured Concurrency (Java Project Loom). 2023.
- [11] M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski. Conflict-Free Replicated Data Types. *SSS*, 2011.
- [12] J.-B. Mouret, J. Clune. Illuminating Search Spaces by Mapping Elites. arXiv:1504.04909, 2015.